Computer Vision

Lecture 4: From Neural Networks to CNNs

Nguyen Manh Toan

Swinburne Vietnam

Week 4

Today's Lecture

1. Recap & Motivation
2. Neural Networks
3. Activation Functions
4. Backpropagation
5. Convolution as a Layer
6. Wrap-Up

Recap & Motivation

Where We Left Off

Last week:

$$s = Wx, \qquad L = \frac{1}{N}\sum_i -\log p_{y_i} + \lambda \|W\|^2, \qquad W \leftarrow W - \eta \nabla_W L$$

What worked

- Fast at test time, no stored training set
- The gradient was clean, cheap, and checkable
- Training loop is used all semester

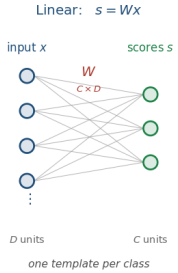
What did not

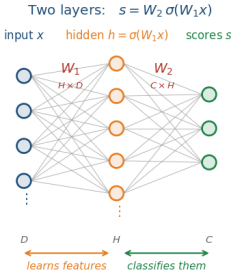
- One template per class — a cat facing left and one facing right must share it
- Cannot separate concentric rings, or XOR
- Operates on raw pixels

Today: Fix both problems — with one nonlinearity, and then with one structural idea.

Neural Networks

Adding a Layer



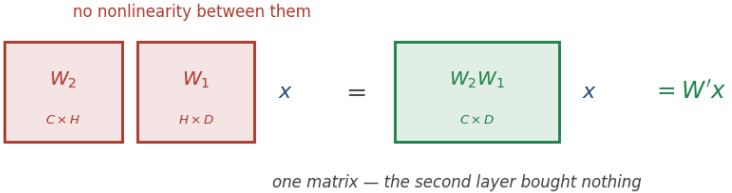


$$s = Wx \longrightarrow s = W_2\,\sigma(W_1 x)$$

- $W_1 \in \mathbb{R}^{H \times D}$, $W_2 \in \mathbb{R}^{C \times H}$
- H is the **hidden size** — typically 100–4096
- σ is a **nonlinear function**, applied elementwise

- The second layer is *last week's classifier*
- The first layer transforms the input
- Three layers: $s = W_3\sigma(W_2\sigma(W_1 x))$, and so on

Without nonlinearity σ, Depth Is Free of Charge

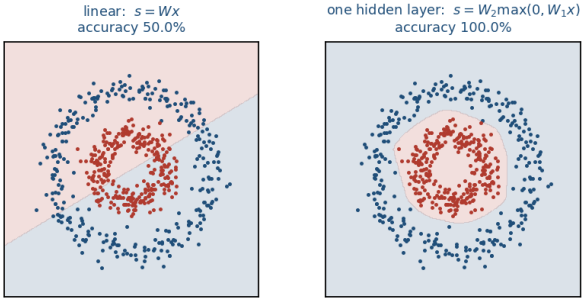


- Two stacked linear layers are *exactly* one linear layer with $W' = W_2 W_1$
- σ is not a detail bolted on for convenience — it is the **only** reason depth exists

Hidden Layer

A two-layer net does exactly what we did by hand last week — transform, then classify — except the transform is **learned**.

A linear model gets one template per class. A two-layer net gets H templates and can **combine** them



The same data, linear vs. one hidden layer

Universal Approximation

The theorem (Cybenko 1989, Hornik 1991)

A network with one hidden layer and a suitable nonlinearity can approximate any continuous function on a compact domain to arbitrary accuracy, given enough hidden units.

What people take from it

"Neural networks can represent anything."

What it actually leaves open

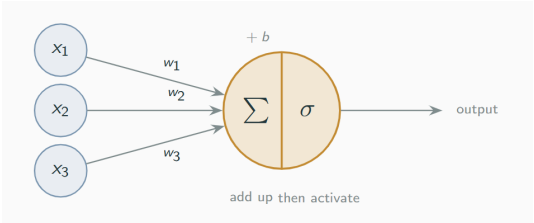
- **How many** hidden units — possibly exponentially many
- Whether SGD can **find** that solution
- Whether it would **generalize** if found

Representation was never the bottleneck. What matters is finding a good function from finite data — which is why architecture, not width, is the subject of Week 5.

The Neuron Analogy — Handle With Care

A unit computes $h_j = \sigma\big(\sum_d W_{jd} x_d + b_j\big)$, which is loosely:

- dendrites collect signals (x_d)
- synapses weight them (W_{jd})
- the cell body sums them
- the axon fires if the sum is large enough (σ)



Useful for intuition and for naming things. Do not use it as an argument that a network will behave like a brain.

Where the analogy fails

Real neurons spike in time, have dendritic nonlinearities, dozens of neurotransmitter types, and are not trained by gradient descent — there is no known biological mechanism for backpropagation.

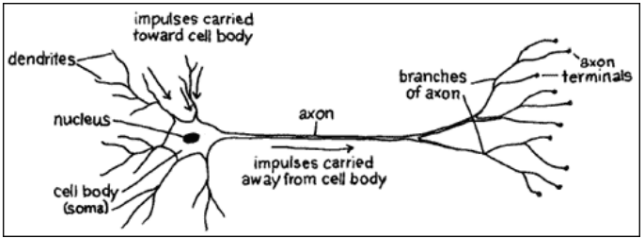


Figure: McCulloch-Pitts (1943)

Activation Functions

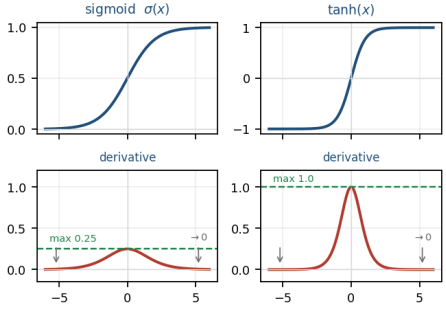
The Classic Choices

$$\sigma(x) = \frac{1}{1+e^{-x}} \qquad \sigma' = \sigma(1-\sigma) \le \tfrac{1}{4}$$

- Squashes to $(0, 1)$; historically read as a firing rate
- **Saturates**: for $|x| > 5$ the gradient is essentially zero
- Not zero-centred — all gradients into a unit share a sign

$$\tanh(x) = 2\sigma(2x) - 1 \qquad \tanh' = 1 - \tanh^2 \le 1$$

- Zero-centred — strictly better than sigmoid, but it still saturates at both ends



Both derivatives vanish in the tails

ReLU and Its Relatives

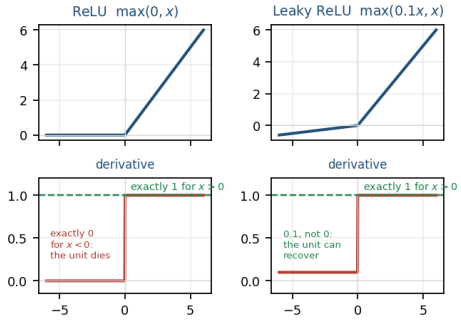
$$\mathrm{ReLU}'(x) = \begin{cases} 1 & x > 0 \\ 0 & x < 0 \end{cases}$$

Why it took over

- **No saturation** for $x > 0$ — the gradient is exactly 1
- Cheap, and converges several times faster than tanh (AlexNet, 2012)

The failure: dying ReLU

- A unit negative for *every* example gets zero gradient forever
- **Leaky ReLU**'s slope lets it recover; **ELU/GELU** smooth the corner



Neither saturates for $x > 0$

Why Saturation Kills Deep Networks

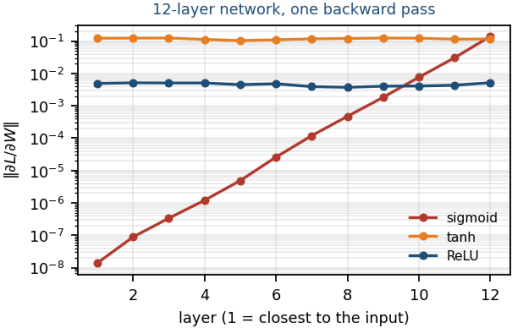
Gradients multiply as they pass backwards through layers:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial h_n} \underbrace{\sigma'(\cdot)\, W_n \cdots \sigma'(\cdot)\, W_1}_{n \text{ factors}}$$

With sigmoid, every factor carries $\sigma' \leq \frac{1}{4}$. After n layers the gradient is scaled by at most 4^{-n}:

- $n = 5$: $\sim 10^{-3}$
- $n = 10$: $\sim 10^{-6}$
- $n = 20$: numerically zero

The early layers stop learning entirely — the **vanishing gradient** problem



Gradient magnitude by layer, measured on a 12-layer net

Backpropagation

The Problem

We need $\nabla_{W_1} L$ and $\nabla_{W_2} L$ for $L\big(W_2\,\sigma(W_1 x)\big)$. Deriving the loss by hand:

- For two layers: tedious
- For fifty layers: impossible
- Redo it every time the architecture changed

The idea

Express the computation as a **graph** of simple operations. Each node needs to know only how to differentiate *itself*. The chain rule assembles the rest.

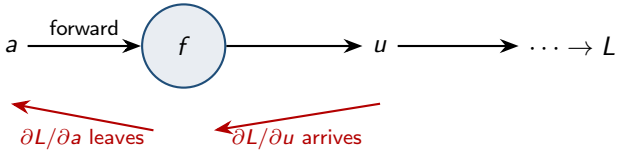
The payoff

One backward pass gives *every* partial derivative, at roughly the cost of one forward pass — regardless of how many parameters there are.

Variable

The loss is a function of the **weights**. The input x and the label y are *constants* — we differentiate with respect to W, never with respect to the data.

The Chain Rule, as a Local Rule



Write a for whatever goes *into* a node and u for whatever comes *out*. Then every node performs the same two-step operation:

$$\underbrace{\frac{\partial L}{\partial a}}_{\text{downstream}} = \underbrace{\frac{\partial L}{\partial u}}_{\text{upstream, given}} \cdot \underbrace{\frac{\partial u}{\partial a}}_{\text{local, known}}$$

Why the letters are deliberately anonymous

a is a *slot*, not a fixed quantity: at one node it is the input image, at the next an activation, at a third a **weight**. Only the last kind matters for learning — but the node cannot tell the difference, and does not need to. That indifference is what makes one backward pass produce every gradient at once.

Example

$$f(a,b,c) = (a+b)\cdot c, \qquad a=-2,\ b=5,\ c=-4$$

Forward: $q = a + b = 3,\ f = qc = -12$

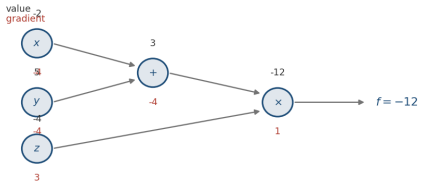
Local gradients

$$\frac{\partial q}{\partial a} = \frac{\partial q}{\partial b} = 1, \qquad \frac{\partial f}{\partial q} = c, \quad \frac{\partial f}{\partial c} = q$$

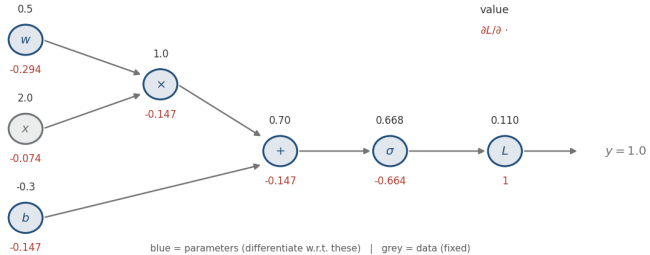
Backward (start from $\partial f/\partial f = 1$)

$$\frac{\partial f}{\partial c} = q = 3, \qquad \frac{\partial f}{\partial q} = c = -4$$

$$\frac{\partial f}{\partial a} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial a} = -4, \qquad \frac{\partial f}{\partial b} = -4$$



On an Actual Weight



One neuron, squared error: $L = \big(\sigma(wx + b) - y\big)^2$, with $w = 0.5$, $x = 2$, $b = -0.3$, $y = 1$.

Backward, right to left

$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y) = -0.664$$

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial \hat{y}}\,\sigma(1 - \sigma) = -0.147$$

Into the parameters (multiply gate: swap)

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot x = -0.294, \qquad \frac{\partial L}{\partial b} = \frac{\partial L}{\partial z} = -0.147$$

$\partial L/\partial x$ exists too, but we never use it — x is data.

Gate Patterns

Add gate

$$u = a + b$$

$\partial u/\partial a = \partial u/\partial b = 1$

A **distributor**: passes the upstream gradient to both inputs unchanged.

Multiply gate

$$u = a \cdot b$$

$\partial u/\partial a = b$

A **swapper**: each input receives the upstream gradient times the *other* input.

Max gate

$$u = \max(a, b)$$

A **router**: the full gradient goes to the larger input, and zero to the other.

Consequence

In the multiply gate, a very large input sends a very large gradient to the *other* branch. This is precisely why unnormalized data destabilizes training — and one reason Lecture 3 insisted on preprocessing.

When a Value Is Used Twice

If a value a feeds several downstream nodes $u_1, u_2, \ldots$, its gradients **add**:

$$\frac{\partial L}{\partial a} = \sum_k \frac{\partial L}{\partial u_k} \frac{\partial u_k}{\partial a}$$

- This is the multivariable chain rule
- Frameworks *accumulate* into `.grad` rather than overwrite it
- Must call `optimizer.zero_grad()` every iteration

A common PyTorch bug

Forget `zero_grad()` and your gradients silently accumulate across iterations. Training does not crash — it just quietly does the wrong thing.

Vector and Matrix Form

With vectors, the local derivative is a **Jacobian** $\partial u/\partial a \in \mathbb{R}^{m\times n}$ — but we almost never build it.

For an elementwise operation the Jacobian is diagonal, so the backward pass is just a product. For an activation $H = \sigma(Z)$:

$$\frac{\partial L}{\partial Z} = \frac{\partial L}{\partial H} \odot \sigma'(Z)$$

For the matrix product $Z = XW^\top$ with $X \in \mathbb{R}^{N\times D}$, $W \in \mathbb{R}^{H\times D}$:

$$\frac{\partial L}{\partial W} = \left(\frac{\partial L}{\partial Z}\right)^\top X, \qquad \frac{\partial L}{\partial X} = \frac{\partial L}{\partial Z}\, W$$

The shape trick

You rarely need to re-derive these. The gradient of a quantity has the **same shape** as the quantity, and there is usually exactly one way to multiply the available matrices to get that shape.

Check shapes first; it catches most errors before you run anything.

A 4096×4096 Jacobian would be 67 MB *per example*. Frameworks implement the vector–Jacobian product directly instead.

The Two-Layer Network, End to End

Forward

$$
\begin{aligned}
Z_1 &= XW_1^\top + b_1 \\
H &= \max(0, Z_1) \\
S &= HW_2^\top + b_2 \\
L &= \text{softmax\_loss}(S, y)
\end{aligned}
$$

Backward

$$
\begin{aligned}
\frac{\partial L}{\partial S} &= (P - Y)/N \\
\frac{\partial L}{\partial W_2} &= \left(\frac{\partial L}{\partial S}\right)^\top H, \qquad \frac{\partial L}{\partial b_2} = \sum_{\text{rows}} \frac{\partial L}{\partial S} \\
\frac{\partial L}{\partial H} &= \frac{\partial L}{\partial S} W_2 \\
\frac{\partial L}{\partial Z_1} &= \frac{\partial L}{\partial H} \odot 1[Z_1 > 0] \\
\frac{\partial L}{\partial W_1} &= \left(\frac{\partial L}{\partial Z_1}\right)^\top X, \qquad \frac{\partial L}{\partial b_2} = \sum_{\text{rows}} \frac{\partial L}{\partial Z_1}
\end{aligned}
$$

Every deep network has this pattern, repeated — which is exactly why autograd can automate it.

Convolution as a Layer

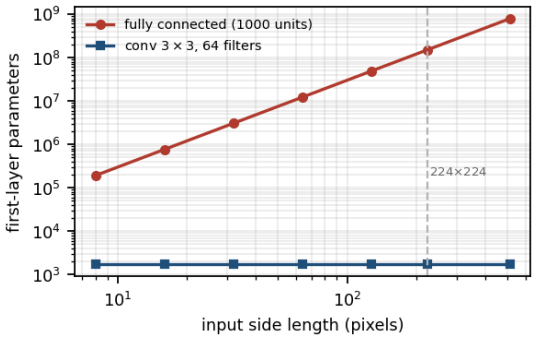
Why Fully Connected Fails on Images

Take a modest $224 \times 224 \times 3$ image into a hidden layer of 1000 units:

$$150{,}528 \times 1000 \approx 1.5 \times 10^8 \text{ parameters}$$

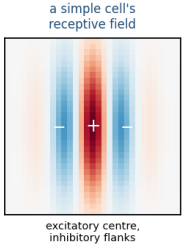
in the first layer alone.

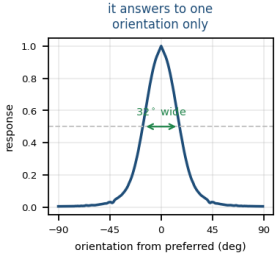
- Far more parameters than training images
- Flattening **destroys spatial structure**: neighbouring pixels become arbitrary, unrelated coordinates
- No translation equivariance — a cat learned in the top-left teaches the model nothing about a cat in the bottom-right

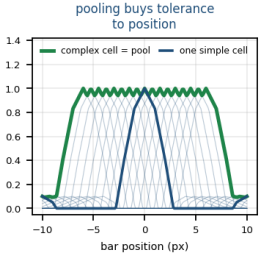


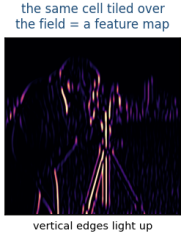
Parameters vs. input size, FC against conv

Visual Cortex









Hubel and Wiesel (1959–62) recorded single cells in cat V1:

- each cell watches a small **receptive field**, not the whole retina
- **simple cells** fire for an oriented edge at a specific place
- **complex cells** keep firing as that edge *moves*
- the same cell type repeats across the visual field

As an architecture

Local window ⇒ small kernel.
Same cell everywhere ⇒ **weight sharing**.
Simple → complex ⇒ **conv** → **pool**.
Stack the pair ⇒ a **hierarchy**.

Fukushima's Neocognitron (1980) copied this literally — its layers are named S and C after simple and complex cells.

Two Priors

Locality

A pixel is related to its neighbors, not to a pixel 200 rows away. So a unit should look at a small **local window**, not the whole image.

Translation equivariance

An edge detector is useful everywhere in the image. So the *same weights* should be applied at every position — **parameter sharing**.

Shift the input, and the output shifts identically.

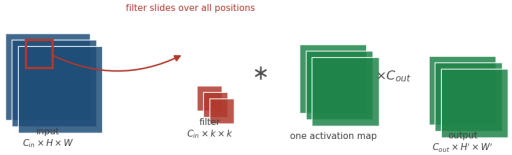
A layer that is local, shift-equivariant, and linear *is* a convolution (Lecture 2). The only new ingredient is that the kernel is learned.

The Convolution Layer

Input $C_{\text{in}} \times H \times W$, kernel $C_{\text{out}} \times C_{\text{in}} \times k \times k$:

$$Z_{c,i,j} = \sum_{c'=1}^{C_{\text{in}}} \sum_{u=1}^{k} \sum_{v=1}^{k} W_{c,c',u,v}\, X_{c',\, i+u,\, j+v} \;+\; b_c$$

- Each filter spans **all** input channels — it is a small 3D volume, not a 2D square
- Each filter produces one 2D **activation map**; C_{out} filters stack into the output
- Parameters: $C_{\text{out}} \cdot C_{\text{in}} \cdot k^2 + C_{\text{out}}$ — independent of H and W



One filter sweeps the input; C_{out} filters give C_{out} maps

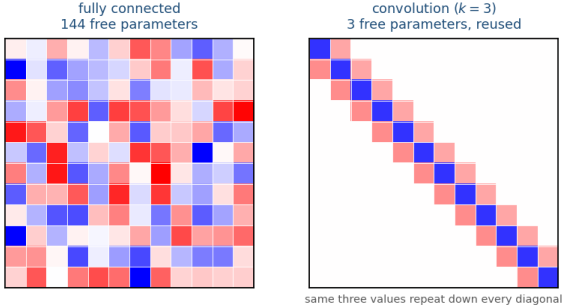
It is really cross-correlation — as we established in Lecture 2, and as `Conv2d` implements.

Convolution as a Constrained Fully Connected Layer

Write the convolution as a matrix multiplication
$\Longrightarrow$ It is forced into a particular shape:

- **Sparse**: each output touches only k inputs, so almost every entry is zero
- **Tied**: the surviving entries repeat down the diagonals — the same k numbers, over and over

So a conv layer is a fully connected layer with most weights set to zero and the rest forced to be equal. It is *strictly less expressive*.



Dense weight matrix vs. the banded, tied matrix of a convolution

Constraints that match the problem are how to win with limited data. This one encodes "images are local and translation-equivariant".

Output Size Arithmetic

$$H_{\text{out}} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1$$

- **Padding** p: 'same' padding is $p = (k-1)/2$ for odd k, which preserves the spatial size
- **Stride** s: $s = 2$ halves the resolution — a cheaper alternative to pooling
- **Dilation** d: insert $d-1$ gaps between kernel elements; effective kernel $k' = d(k-1)+1$. Grows the receptive field without extra parameters

Example

For the first ResNet layer:
$H \times H = 224 \times 224$, $k = 7$, $s = 2$, $p = 3$:

$$H_{\text{out}} = \left\lfloor \frac{224 + 6 - 7}{2} \right\rfloor + 1 = 112$$

Get this formula wrong and the tensor shapes will not match three layers later. The error is much harder to read.

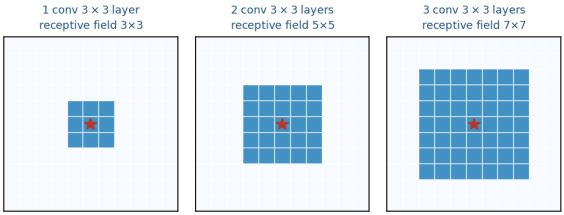
Receptive Fields

The **receptive field** of a unit is the region of the *input* that can influence it. Stacking 3×3 convolutions grows it linearly:

- 1 layer: 3×3
- 2 layers: 5×5
- 3 layers: 7×7

Three 3×3 layers see the same region as one 7×7 layer, but with $3(9C^2) = 27C^2$ parameters instead of $49C^2$ — and two extra nonlinearities.

That argument is the whole of VGG. Pooling and stride grow the field much faster, multiplicatively.



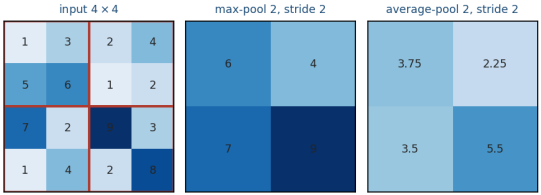
Receptive field growth with depth

Pooling

Downsample each activation map independently. Typically 2×2 with stride 2:

$$\text{max-pool}(X)_{i,j} = \max_{u,v \in 2\times 2} X_{2i+u,\, 2j+v}$$

- Has **no parameters**
- Cuts compute by $4\times$
- Grows the receptive field multiplicatively
- A little *local* translation invariance
- Backward pass: route the gradient to the argmax, zero elsewhere (the max gate from earlier)



Max vs. average pooling Modern nets often drop pooling for strided convolution; global average pooling before the classifier is now standard.

Backward Through a Convolution

A conv layer is linear, so the same two rules from the backprop section apply — and both results turn out to be convolutions themselves.

Gradient w.r.t. the weights

$$\frac{\partial L}{\partial W} = X \star \frac{\partial L}{\partial Z}$$

Correlate the input with the upstream gradient. Every position where the filter was applied contributes — exactly the "gradients add when a value is reused" rule, applied $H' \times W'$ times.

Gradient w.r.t. the input

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Z} * \tilde{W}$$

A full convolution with the kernel flipped in both axes — the "transposed convolution" you will meet again in segmentation and generative models.

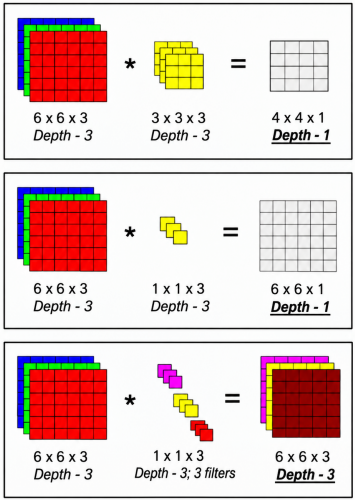
Parameter sharing is why the weight gradient sums over all spatial positions. One filter, many uses, many contributions.

The 1×1 Convolution

A 1×1 kernel looks at a single spatial position — but at **all** channels:

$$Z_{c,i,j} = \sum_{c'} W_{c,c'} X_{c',i,j}$$

- It is a fully connected layer applied *independently at every pixel*
- Its job is **channel mixing** and dimensionality reduction: $256 \to 64$ channels costs 256×64 parameters and no spatial extent
- Cheap way to add a nonlinearity



Why Deep Rather Than Wide?

Universal approximation says one hidden layer is enough. Practice says otherwise. The reason is **composition**.

- A shallow, wide net must represent every pattern independently. A deep net **reuses** its earlier features: one edge detector serves every object class above it
- Certain functions need exponentially many units at depth 2, but only polynomially many at depth k
- Each layer adds a nonlinearity, so the number of linear regions the network can carve out grows multiplicatively with depth

The empirical record

- AlexNet (2012): 8 layers, 16.4% top-5 error
- VGG (2014): 19 layers, 7.3%
- ResNet (2015): 152 layers, 3.6%

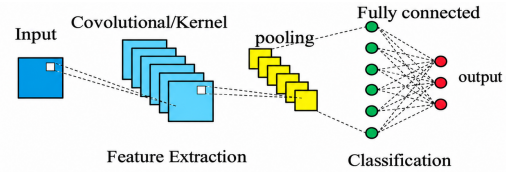
Depth won every time — but only once people worked out how to *train* it. That story is Week 5.

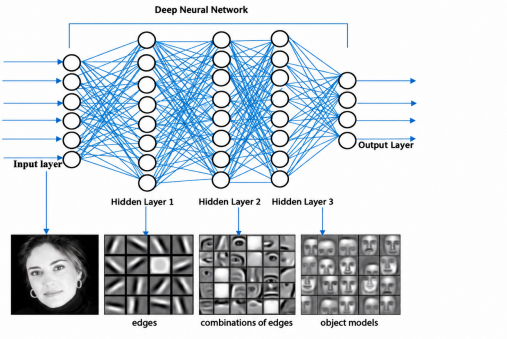
What the Filters Learn

Train a CNN on images, then look at the first-layer kernels.

- Oriented edges at several scales and orientations
- Color-opponent blobs
- Which is to say: **Gabor filters** — the operators we designed by hand in Lecture 2

This emerges from gradient descent on a classification loss, and it reproduces both the classical CV toolbox and the receptive fields Hubel and Wiesel measured in cat visual cortex.

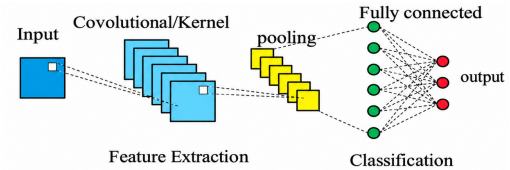


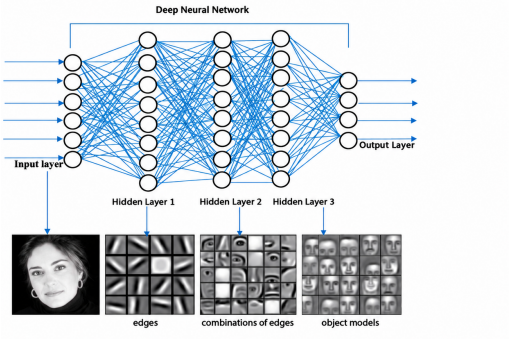


Depth Builds a Hierarchy

- **Early layers**: edges, colours, simple textures — small receptive field, generic
- **Middle layers**: corners, motifs, textures, object parts
- **Late layers**: whole objects and scene-level structure — large receptive field, task-specific

This is why **transfer learning** works: the early layers of a network trained on ImageNet are useful for essentially any vision task. Almost every project in this course will start from pretrained weights.





A First CNN

Layer	Output	Params
input	$1 \times 28 \times 28$	0
conv 3×3, 16, pad 1	$16 \times 28 \times 28$	160
ReLU + maxpool 2	$16 \times 14 \times 14$	0
conv 3×3, 32, pad 1	$32 \times 14 \times 14$	4,640
ReLU + maxpool 2	$32 \times 7 \times 7$	0
flatten	1568	0
fully connected	10	15,690
total		**20,490**

Read the parameter column

The convolutions do the real work with 4,800 parameters. The single fully connected layer at the end uses three times as many.

That imbalance is why modern architectures replace the final FC layers with **global average pooling**.

The pattern `[conv` $\rightarrow$ `ReLU` $\rightarrow$ `pool]` $\times$ `N` $\rightarrow$ `FC` is LeNet-5 (1998), and it is still the skeleton of everything in Week 5.

Wrap-Up

Summary

- **One nonlinearity** turns a stack of matrices into a model that can learn its own features. Without it, depth collapses to a single linear layer
- **ReLU** does not saturate. Sigmoid multiplies at most $\frac{1}{4}$ per layer, which is fatal at depth
- **Backpropagation** is the chain rule applied locally on a graph. Each node needs only its own derivative; gradients *add* where a value is reused
- **Convolution** is the layer you get by demanding locality and translation equivariance — Lecture 2's operator with learned kernels
- Parameters do not grow with image size; receptive fields grow with depth; first-layer filters converge to **Gabor filters** on their own

Next week: CNN architectures — AlexNet, VGG, GoogLeNet, ResNet, and why depth needs skip connections.

Questions?

Reading & Reference

- Stanford CS231n notes: *Neural Networks 1–3*, *Backpropagation*, *Convolutional Networks*
- LeCun et al., "Gradient-Based Learning Applied to Document Recognition", *Proc. IEEE* 1998 (LeNet-5)
- Krizhevsky, Sutskever & Hinton, "ImageNet Classification with Deep CNNs", *NeurIPS* 2012 (AlexNet)
- Rumelhart, Hinton & Williams, "Learning representations by back-propagating errors", *Nature* 1986
- Hubel & Wiesel, "Receptive fields of single neurones in the cat's striate cortex", *J. Physiol.* 1959

Hands-On (Last 30 Minutes)

Notebook: `lecture04_handson.ipynb`

- Backprop by hand through $f = (a + b)c$, then check every partial numerically
- Watch a two-layer net fail on `make_circles` with the nonlinearity removed
- Implement the full forward and backward pass in NumPy — and gradient-check both layers
- Train it; visualize what the hidden units learned
- Measure the vanishing gradient: sigmoid vs. ReLU, gradient norm by layer
- Reproduce the whole thing in PyTorch and confirm autograd matches your algebra
- Build a conv layer by hand, verify against `F.conv2d`, and check the output-size formula
- Train a small CNN on digits and look at the first-layer kernels